

# RISC-V External Debug Security Specification

Version 0.7.7, 2026-11-03: Updated Release Candidate for ARC Review

# Table of Contents


Preamble ..... 1
Copyright and license information ..... 3
Contributors ..... 5
1. Introduction ..... 7
1.1. Control Hierarchy ..... 7
1.1.1. External Debug Security Control ..... 7
1.1.2. Trace Security Control ..... 9
1.2. Terminology ..... 10
2. External Debug Security Threat Model ..... 11
3. Secure Debug and Secure Trace (ISA Extensions) ..... 13
3.1. External Debug Security Extensions ..... 13
3.1.1. External Debug Security Control States and CSRs ..... 13
3.1.2. Debug Operations ..... 15
3.1.3. Debug Access Privilege ..... 15
3.1.4. Allowed Resume Privilege Modes ..... 15
3.1.5. M-mode External Debug Security Extension (Smmedbgsec) ..... 16
3.1.6. S/HS-mode External Debug Security Extension (Smsedbgsec) ..... 17
`sdcsr` and `sdpc` ..... 18
Debug Access Privilege for Memory in Smsedbgsec ..... 19
3.1.7. VS-mode External Debug Security Extension (Smvsedbgsec) ..... 19
`sdcsr` and `sdpc` in VS-mode ..... 20
Debug Access Privilege for Memory in Smvsedbgsec ..... 20
3.1.8. U-mode and VU-mode External Debug Security Extension (Smuedbgsec) ..... 20
U-mode External Debug Control ..... 21
VU-mode External Debug Control ..... 21
`udcsr` and `udpc` ..... 22
3.2. Trace Security Extensions ..... 22
3.2.1. Trace Security Control States ..... 22
3.2.2. M-mode Trace Security Extension (Smmetrcsec) ..... 23
3.2.3. S/HS-mode Trace Security Extension (Smsetrcsec) ..... 24
3.2.4. VS-mode Trace Security Extension (Smvsetrcsec) ..... 24
3.2.5. U-mode and VU-mode Trace Security Extension (Smuetrcsec) ..... 24
4. Smtdeleg and Sstcfg (ISA Extensions) ..... 25
4.1. Supervisor and Virtual Supervisor Trigger Access ..... 25
4.2. Trigger State Access Modifications ..... 25
4.3. DMODE ..... 26
4.4. Supervisor and Virtual Supervisor Trigger State Access Control ..... 27
4.5. Dependencies ..... 27
4.6. Summary of New State ..... 27
5. Sucsrind (ISA Extension) ..... 29
5.1. User-level CSRs ..... 29
5.2. Virtualization ..... 30
5.3. Access Control by the State-Enable CSRs ..... 30

5.4. Dependencies ... 30
5.5. Summary of New State ... 31
**6. Sutcfg (ISA Extension) ... 33**
6.1. User Trigger Access ... 33
6.2. DMODE ... 33
6.3. User Trigger State Access Control ... 33
6.4. Summary of New State ... 34
**7. Debug Module Security (non-ISA) Extension ... 35**
7.1. External Debug Security Extensions Discovery ... 35
7.2. Halt ... 35
7.3. Reset ... 35
7.4. Keepalive ... 35
7.5. Abstract Commands ... 36
7.5.1. Relaxed Permission Check `relaxedpriv` ... 36
7.5.2. Address Translation AAMVIRTUAL ... 36
7.5.3. Quick Access ... 36
7.6. System Bus Access ... 36
7.7. Security Fault Error Reporting ... 36
7.8. Platform Debug Security Enable ... 37
7.9. Authorization Handoff ... 37
7.10. Update of Debug Module Registers ... 38
**Appendix A: Valid Extension Combinations ... 39**
A.1. External Debug Security Extension Combinations ... 39
A.2. Trace Security Extension Combinations ... 39
**Appendix B: Software Debug/Trace Security Policy Management ... 41**
B.1. M-mode Management ... 41
B.2. S/HS-mode Management ... 41
B.3. VS-mode Management ... 41
**Appendix C: Execution-Based Implementation with Sdsec ... 43**
**Bibliography ... 45**

# Preamble



*This document is in the Development state*

*Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.*

# Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2023-2024 by RISC-V International.

# Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order): Allen Baum, Aote Jin (editor), Beeman Strong, Erik Kraft, Gokhan Kaplayan, Greg Favor, Iain Robertson, Joe Xie (editor), Paul Donahue, Ravi Sahita, Robert Chyla, Thomas Roecker, Tim Newsome, Ved Shanbhogue, Vicky Goode

# Chapter 1. Introduction

Debugging and tracing are essential for developers to identify and rectify software and hardware issues, optimize performance, and ensure robust system functionality. The debugging and tracing extensions in the RISC-V ecosystem play a pivotal role in enabling these capabilities, allowing developers to monitor and control the execution of programs during the development, testing, and production phases. However, the current RISC-V Debug specification grants the external debugger the highest privilege in the system, regardless of the privilege level at which the target system is running, and tracing is also allowed in all privilege modes. This leads to privilege escalation and information leakage issues when multiple actors are present.

This specification defines Debug Module Security Extension (non-ISA extension) and Secure Debug and Secure Trace ISA extensions (collectively referred to as Sdsec) to address the above security issues in the current *The RISC-V Debug Specification* [1] and trace specifications [2] [3]. It also defines the trigger delegation extensions Smtdeleg, Sstcfg, Sucsrind, and Sutcfg, which let software and external debuggers with less than M-mode privilege access Sdtrig triggers.

At a high level, the Debug Module Security Extension provides a platform-level control state, `pdbgsecen`, to enable or bypass the debug security constraints. The Secure Debug and Secure Trace ISA extensions provide per-hart debug and trace control states, including platform-controlled `mdbgen` and `mtrcen` states for M-mode and lower-privilege control states in `mdtcfg`. Together, these control states determine whether external debug or trace is allowed at the hart's current privilege mode. `pdbgsecen` does not control trace. Implementation of either ISA family is optional even when the corresponding external debug or trace facility is present.

A summary of the changes introduced by *The RISC-V External Debug Security Specification* follows.

- **Per-Hart Debug Control:** Introduce per-hart states to control whether external debug is allowed for each privilege mode.
- **Per-Hart Trace Control:** Introduce per-hart states to control whether tracing is allowed for each privilege mode.
- **Platform debug security enable:** Add a platform state to enable the security constraints or bypass them for backward compatibility.
- **Debug Mode entry:** An external debugger can only halt the hart and enter Debug Mode when external debug is allowed in the current privilege mode.
- **Memory Access:** Memory access from a hart's point of view, using the Program Buffer or an Abstract Command, must be checked by the hart's memory protection mechanisms as if the hart is running at debug access privilege level; memory access from the Debug Module using System Bus Access (SBA) must be checked by a system memory protection mechanism, such as IOPMP or RISC-V Worlds.
- **Register Access:** Register access using the Program Buffer or an Abstract Command works as if the hart is running at debug access privilege level instead of the M-mode privilege level. The debug CSRs (`dcsr` and `dpc` ) are shadowed in lower privilege modes.

## 1.1. Control Hierarchy

Operationally, `pdbgsecen` selects between backward-compatible debug behavior and the debug security constraints defined by this specification. When those constraints are enabled, each hart evaluates its debug control states for its current privilege mode. Debug operations are permitted only when the corresponding debug control state allows access at that mode; otherwise the request is rejected or remains pending. Trace output is independent of `pdbgsecen` and is enabled only for modes allowed by the trace control states.

### 1.1.1. External Debug Security Control

The debug security policy for a hart is enforced through a hierarchical control model using the platform-controlled `mdbgen` state for M-mode and optional lower-privilege control states (`SEDBGEN`, `VSEDBGEN`, `UEDBGEN`, `VUEDBGEN`) in the `mdtcfg` CSR. The `mdbgen` state can be implemented as an input signal to the hart and managed through platform-specific mechanisms such as Debug Module registers, a Root of Trust (RoT) handshake, or lifecycle fuses. The `pdbgsecen` platform control state enables the debug security policy at the platform level; when `pdbgsecen` is 0, the platform provides unrestricted debug access for all privilege modes.

The debug security control logic validates all debug requests and triggers whose `ACTION`=1 against the hart's current privilege level and the configured debug security control states. The validation procedure involves two steps: (1) checking `pdbgsecen` to determine whether the debug security policy is enabled and (2) checking the hierarchical control states to determine whether external debug is allowed at the hart's current privilege level. Debug requests that fail validation are either dropped or kept pending until the hart transitions to a debug-allowed privilege mode. External Debug Security Extensions defines the detailed behavior.

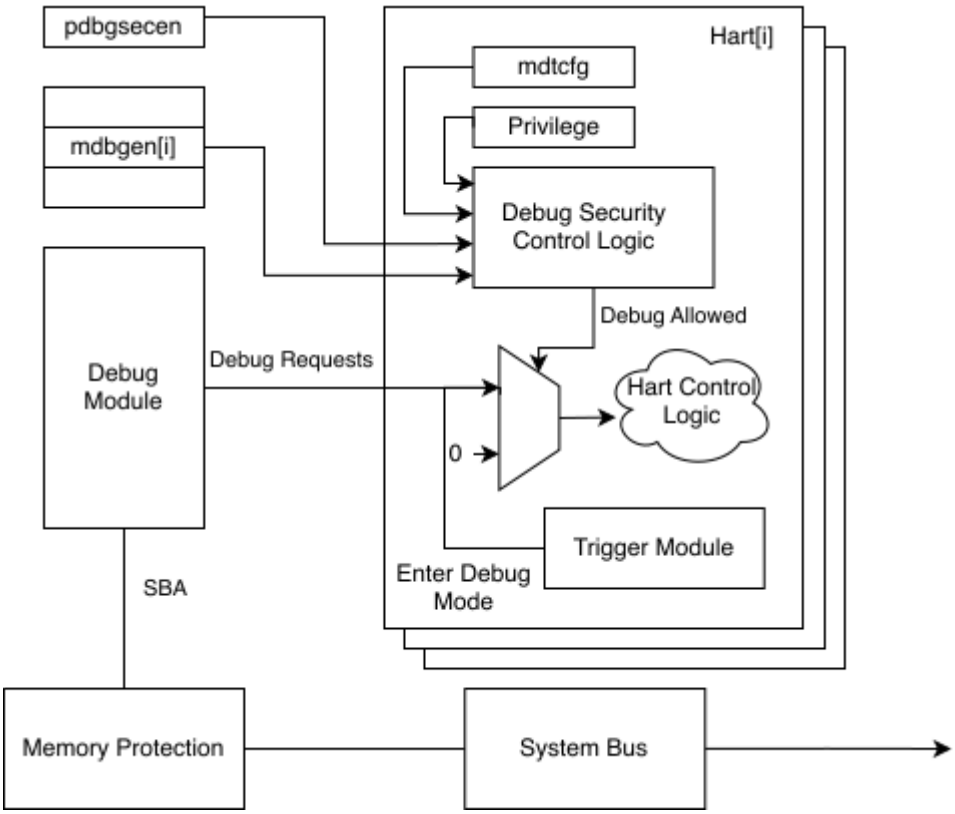



*Figure 1. The debug security control for one hart*

The hierarchical control applies as follows for a single hart:

- When `pdbgsecen`=0, external debug is allowed for all privilege modes.
- When `pdbgsecen`=1 and `mdbgen`=1, external debug is allowed for all privilege modes.
- When `pdbgsecen`=1, `mdbgen`=0, and `SEDBGEN`=1 (if implemented), external debug is allowed for S/HS-mode and lower privilege modes.
- When `pdbgsecen`=1, `mdbgen`=0, `SEDBGEN`=0, and `VSEDBGEN`=1 (if implemented), external debug is allowed for VS-mode and VU-mode.
- When `pdbgsecen`=1, `mdbgen`=0, `SEDBGEN`=0, and `UEDBGEN`=1 (if implemented), external debug is allowed for U-mode. `VSEDBGEN` and `VUEDBGEN` do not affect U-mode debug permission.

- When `pdbgsecen`=1, `mdbgen`=0, `SEDBGEN`=0, `VSEDBGEN`=0, and `VUEDBGEN`=1 (if implemented), external debug is allowed for VU-mode. `UEDBGEN` does not affect VU-mode debug permission.
- External debug is disallowed in a privilege mode when none of the control states applicable to that mode allow debug.

The debug security control logic must ensure that debug requests are validated and executed at the same privilege level and with the same debug security control states to prevent time-of-check to time-of-use (TOCTOU) vulnerabilities. Failing to do so may allow debug requests to bypass security controls.



*Self-hosted debugging is also commonly used for application-level debugging, providing a pure software-based solution that does not require an external debugger. It is orthogonal to external debugging and is not affected by the debug security controls.*

### 1.1.2. Trace Security Control

The trace security policy for a hart is enforced through a hierarchical control model using the platform-controlled `mtrcen` state for M-mode and optional lower-privilege control states (`SETRCEN`, `VSETRCEN`, `UETRCEN`, `VUETRCEN`) in the `mdtcfg` CSR. The `mtrcen` state can be implemented as an input signal to the hart and managed through platform-specific mechanisms such as a Root of Trust (RoT) handshake or lifecycle fuses. Trace has no privilege mode above M-mode, so this specification does not introduce a platform-level enable analogous to `pdbgsecen`; `pdbgsecen` does not control trace.

The security control logic determines whether trace is allowed based on the hart's current privilege level and the configured trace security control states. The `sec_inhibit` sideband signal to the trace encoder is asserted when trace is not allowed; equivalently, `sec_inhibit` is the inverse of the trace-allowed decision shown in the reference control logic. Trace Security Extensions defines the detailed behavior.

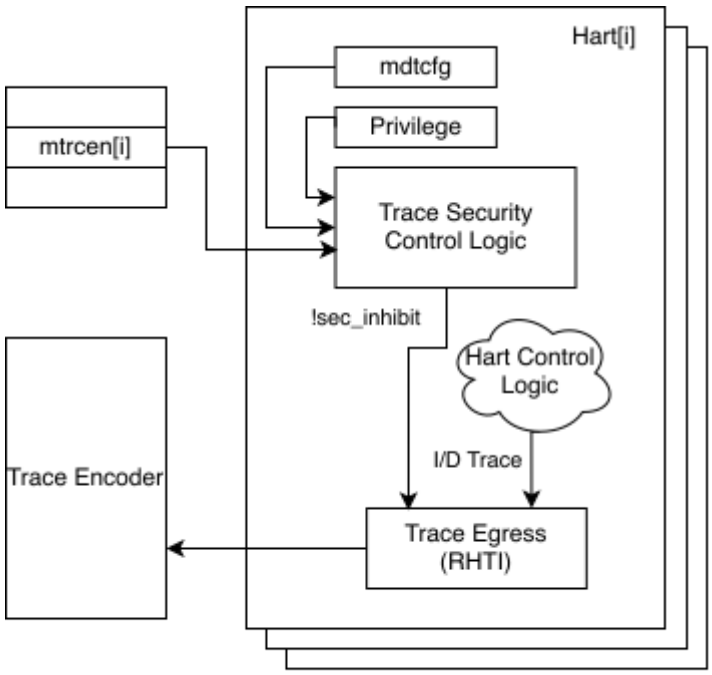



*Figure 2. The trace security control for one hart*

The hierarchical control applies as follows for a single hart:

- When `mtrcen`=1, trace is allowed for all privilege modes.
- When `mtrcen`=0 and `SETRCEN`=1 (if implemented), trace is allowed for S/HS-mode and lower privilege

modes; trace is inhibited for M-mode.

- When `mtrcen`=0, `SETRCEN`=0, and `VSETRCEN`=1 (if implemented), trace is allowed for VS-mode and VU-mode; trace is inhibited for M-mode and S/HS-mode.
- When `mtrcen`=0, `SETRCEN`=0, and `UETRCEN`=1 (if implemented), trace is allowed for U-mode. `VSETRCEN` and `VUETRCEN` do not affect U-mode trace permission.
- When `mtrcen`=0, `SETRCEN`=0, `VSETRCEN`=0, and `VUETRCEN`=1 (if implemented), trace is allowed for VU-mode. `UETRCEN` does not affect VU-mode trace permission.
- When all trace control states are 0, trace is completely inhibited for all privilege modes.

## 1.2. Terminology

| | |
|---|---|
| Abstract command | A high-level Debug Module operation used to interact with and control harts |
| Debug Access Privilege | The privilege level with which an Abstract Command or instruction in the Program Buffer accesses hardware resources |
| Debug Mode | An additional privilege mode to support off-chip debugging |
| Hart | A RISC-V hardware thread |
| IOPMP | Input-Output Physical Memory Protection unit |
| M-mode | The highest privileged mode in the RISC-V privilege model |
| PMA | Physical Memory Attributes |
| PMP | Physical Memory Protection unit |
| Program buffer | A mechanism that allows the Debug Module to execute arbitrary instructions on a hart |
| RISC-V Worlds | A system-level isolation mechanism that constrains physical-address access by harts and other bus initiators |
| Trace encoder | A piece of hardware that takes in instruction execution information from a RISC-V hart and transforms it into trace packets |

# Chapter 2. External Debug Security Threat Model

Modern SoC development involves several different actors who may not trust each other, resulting in the need to isolate actors' assets during the development and debugging phases. The current RISC-V Debug specification [1] grants external debuggers the highest privilege in the system, regardless of the privilege level at which the target system is running. This leads to privilege escalation issues when multiple actors are present.

For example, the owner of an SoC, who needs to debug their firmware, may be able to use the external debugger to bypass PMP (including PMP lock `pmpcfg`.L=1) and MMU protections, attacking Boot ROM or other SoC creator's assets.

Additionally, the RISC-V privilege architecture supports multiple software entities at different privilege levels that may not trust each other. These entities may be isolated from each other by mechanisms such as PMP/IOPMP or MMU, and may need different debug and trace policies. Since the external debugger is granted the highest privilege in the system, a malicious entity at a lower privilege level could compromise higher privilege software with the external debugger. Similarly, trace output from higher privilege execution could leak sensitive information to entities at lower privilege levels if not properly controlled.

# Chapter 3. Secure Debug and Secure Trace (ISA Extensions)

This chapter specifies ISA extensions that add security controls to external debug (Sdext/Sdtrig) [1] and to trace [2] [3]. Sdsec is not an extension of its own; it is the umbrella name for two families of extensions: Secure Debug and Secure Trace.

The Secure Debug family provides hierarchical privilege-based control of external debug. When this family is implemented, the following extensions are defined:

- **Smmedbgsec (Base)**: M-mode external debug control
- **Smsedbgsec (Optional)**: Extends Smmedbgsec to add S/HS-mode external debug control
- **Smvsedbgsec (Optional)**: Adds VS-mode external debug control
- **Smuedbgsec (Optional)**: Adds independent U-mode and VU-mode external debug controls

The Secure Trace family provides hierarchical privilege-based control of trace output. When this family is implemented, the following extensions are defined:

- **Smmetrcsec (Base)**: M-mode trace control
- **Smsetrcsec (Optional)**: Extends Smmetrcsec to add S/HS-mode trace control
- **Smvsetrcsec (Optional)**: Adds VS-mode trace control
- **Smuetrcsec (Optional)**: Adds independent U-mode and VU-mode trace controls

Each family is optional and independent of the other: an implementer may implement the Secure Debug family, the Secure Trace family, or both. Implementing external debug or trace does not by itself require the corresponding family. Implementations may include some or all optional extensions of an implemented family. When an optional extension for a lower privilege mode is implemented, all extensions of the same family for implemented higher privilege modes must also be implemented. The valid combinations are listed in Appendix A. These extensions establish hierarchical controls that can restrict external debug access or trace output to specific privilege levels.

When Sdtrig is implemented, an external debugger whose debug access privilege is less than M-mode can access triggers only through the trigger delegation extensions: Smtdeleg and Sstcfg for S/HS-mode and VS-mode, and Sutcfg with Sucsrind for U-mode and VU-mode. These are distinct extensions. They are not members of either family, and neither family requires them.

## 3.1. External Debug Security Extensions

External debug operations can modify system state and expose sensitive information. The following extensions provide hierarchical privilege-based controls to restrict debug access, preventing unauthorized debugging of higher-privilege software.

### 3.1.1. External Debug Security Control States and CSRs

The following table summarizes the external debug control states and associated CSRs introduced by the Debug Module Security Extension and the Sdsec external debug security extensions. In this specification, *control state* covers platform-level control state, such as `pdbgsecen`, per-hart platform-controlled states, such as `mdbgen`, and CSR fields, such as `mdtcfg.SEDBGEN`.

*Table 1. External Debug Control States and CSRs*

| Extension | External Debug Control State | New CSRs | Scope |
|---|---|---|---|
| Debug Module Security Extension | pdbgsecen | None | Platform/all modes |
| Smmedbgsec | mdbgen | mdtcfg | M-mode |
| Smsedbgsec | mdtcfg.SEDBGEN | sdcsr, sdpc | S/HS-mode |
| Smvsedbgsec | mdtcfg.VSEDBGEN | None | VS-mode |
| Smuedbgsec | mdtcfg.UEDBGEN, mdtcfg.VUEDBGEN | udcsr, udpc | U-mode and VU-mode, independently |

When external debug is allowed for a privilege mode, it is allowed for all lower privilege modes. U-mode and VU-mode are not ordered with respect to each other, so `UEDBGEN` and `VUEDBGEN` control them independently. If an optional debug extension is not implemented, the corresponding control state behaves as read-only 0; for lower-privilege modes, the `mdtcfg` field representing that control state is also read-only 0.

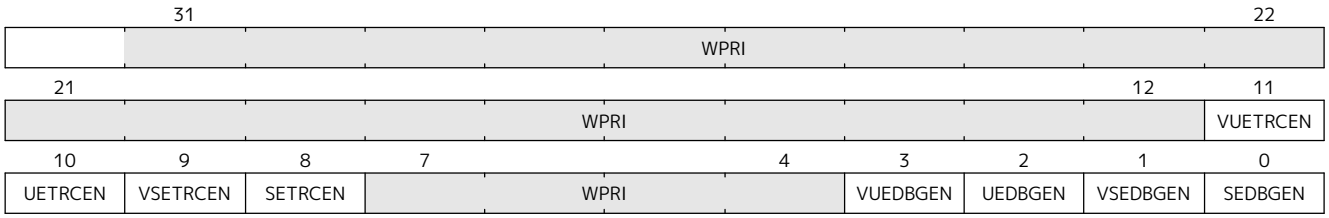
The `mdtcfg` (Machine Debug and Trace Configuration) is a machine-mode read/write CSR (address TBD). It is a single CSR shared by the Secure Debug and Secure Trace families. Debug-related fields (`SEDBGEN`, `VSEDBGEN`, `UEDBGEN`, `VUEDBGEN`) and trace-related fields (`SETRCEN`, `VSETRCEN`, `UETRCEN`, `VUETRCEN`) are independent: an implementation may implement Secure Debug, Secure Trace, or both without coupling the two functions through this CSR.

`mdtcfg` is implemented if Smmedbgsec, Smmetrcsec, or any optional extension in either family is implemented. Fields associated with an unimplemented family or optional extension are read-only zero.

The M-mode external debug and trace enables are the platform-controlled `mdbgen` and `mtrcen` states. They are not fields of `mdtcfg` and are not writable by M-mode software. Writing `mdtcfg` therefore cannot enable or disable M-mode external debug or M-mode trace; M-mode software uses `mdtcfg` only to configure lower-privilege debug and trace enables.

*Table 2. Machine Debug and Trace Configuration CSR*

| Number | Privilege | Width | Name | Description |
|---|---|---|---|---|
| address TBD | MRW | XLEN | `mdtcfg` | Machine debug and trace configuration |




*Register 1:* `mdtcfg` *register*

The following fields in `mdtcfg` provide external debug control states for privilege modes other than M-mode. The trace control states are described in Section 3.2.

*Table 3. Details of external debug control states in* `mdtcfg`

| Field | Description | Access | Reset |
|---|---|---|---|
| SEDBGEN | S/HS-mode external debug enable. When set to 1, external debug is allowed for S/HS-mode, U-mode, VS-mode, and VU-mode when `mdbgen` is 0. Introduced by Smsedbgsec. | WARL | 0 |

| Field | Description | Access | Reset |
|---|---|---|---|
| VSEDBGEN | VS-mode external debug enable. When set to 1, external debug is allowed for VS-mode and VU-mode when `mdbgen`=0 and `SEDBGEN`=0. This field does not control U-mode. Introduced by Smvsedbgsec. | WARL | 0 |
| UEDBGEN | U-mode external debug enable. When set to 1, U-mode external debug is allowed when `mdbgen`=0 and `SEDBGEN`=0. `VSEDBGEN` and `VUEDBGEN` do not affect U-mode permission. Introduced by Smuedbgsec. | WARL | 0 |
| VUEDBGEN | VU-mode external debug enable. When set to 1, VU-mode external debug is allowed when `mdbgen`=0, `SEDBGEN`=0, and `VSEDBGEN`=0. `UEDBGEN` does not affect VU-mode permission. Introduced by Smuedbgsec. | WARL | 0 |



*The bit positions shown above are tentative and have not been officially allocated. The final bit assignments will be determined during the ratification process.*

### 3.1.2. Debug Operations

Chapter 3 of *The RISC-V Debug Specification* [1] outlines all mandatory and optional external debug operations. The operations listed below are affected by the Secure Debug extensions; other operations remain unaffected. In the context of this chapter, **debug operations** refer to those listed below.

Debug operations affected by the Secure Debug extensions:

- Halting the hart to enter Debug Mode
- Executing the Program Buffer
- Serving abstract commands (Access Register, Access Memory)

### 3.1.3. Debug Access Privilege

The **debug access privilege** is the privilege level used for state accesses performed by the Debug Module or through the Program Buffer, including accesses performed by Abstract Commands and Program Buffer instructions. Register and memory accesses use the privilege level derived in Table 4. Attempts to access state that requires a privilege level above the **debug access privilege** fail and set `abstractcs`.CMDERR to 3.

*Table 4. External Debug Configuration and Privilege*

| mdbgen | SEDBGEN | VSEDBGEN | UEDBGEN | VUEDBGEN | Debugged Context | Debug Access Privilege |
|---|---|---|---|---|---|---|
| 1 | Don't care | Don't care | Don't care | Don't care | Any supported mode | M-mode |
| 0 | 1 | Don't care | Don't care | Don't care | S/HS-mode, VS-mode, U-mode, or VU-mode | S/HS-mode |
| 0 | 0 | 1 | Don't care | Don't care | VS-mode or VU-mode | VS-mode |
| 0 | 0 | Don't care | 1 | Don't care | U-mode | U-mode |
| 0 | 0 | 0 | Don't care | 1 | VU-mode | VU-mode |

### 3.1.4. Allowed Resume Privilege Modes

The privilege modes that can be configured in `dcsr`.PRV and `dcsr`.V are determined by the table below. If an unsupported or disallowed value is written, the fields retain legal values.

*Table 5. Allowed Resume Privilege Modes*

| Resume Mode | Required Debug Control State |
|---|---|
| M-mode | `mdbgen`=1 |
| S/HS-mode | `mdbgen`=1, or `mdbgen`=0 and `SEDBGEN`=1 |
| VS-mode | `mdbgen`=1, or `mdbgen`=0 and either `SEDBGEN`=1 or `VSEDBGEN`=1 |
| U-mode | `mdbgen`=1, or `mdbgen`=0 and either `SEDBGEN`=1 or `UEDBGEN`=1 |
| VU-mode | `mdbgen`=1, or `mdbgen`=0 and at least one of `SEDBGEN`, `VSEDBGEN`, or `VUEDBGEN` is 1 |

### 3.1.5. M-mode External Debug Security Extension (Smmedbgsec)

The Smmedbgsec extension is the base extension of the Secure Debug family.



*The `mdbgen` state is a per-hart platform-controlled state that controls whether M-mode external debug capabilities are available. It may be delivered to the hart through various methods, such as a new input port, a handshake with the system Root of Trust (RoT), or other methods. The `mdbgen` state for the Root of Trust (RoT) itself should be managed by SoC hardware, likely depending on lifecycle fusing. The implementation can choose to group several harts together and use one signal to drive their `mdbgen` state or assign each hart its own dedicated state. For example, a homogeneous computing system can use a signal to drive all `mdbgen` states to enforce a unified debug policy across all harts.*

When `mdbgen` is set to 1, debug is allowed for M-mode and the following rules apply:

- The debug access privilege for the hart is M-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with M-mode privilege.
- The debug operations are allowed when the hart executes in any privilege mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs`.CMDERR to 3.
- The allowed resume privilege modes include M-mode and all lower privilege modes.

When `mdbgen` is set to 0 and the hart is running in M-mode, external debug is disallowed for M-mode:

- The hart will not enter Debug Mode:
  - Halt requests will remain pending until external debug is allowed.
  - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
  - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire.
- Accesses to M-mode accessible CSRs (e.g., `dcsr`, `dpc`, and `dscratch0/1`) will fail and `abstractcs`.CMDERR is set to 3 (exception).
- The configuration in `dcsr` will be ignored as if it were 0.
- `DMODE` is read/write for accesses from M-mode when Sdtrig is implemented.



*M-mode is given write access to DMODE when `mdbgen` is 0 to allow M-mode software to save*

*and restore trigger context at S/HS-mode software boundaries. This capability is necessary to prevent triggers from serving as a side-channel attack vector against S/HS-mode software where external debugging is disallowed. Without this, an external debugger could indirectly observe states (such as PC and data addresses) from the debug-disallowed software, potentially leading to information leakage or Denial of Service (DoS). By enabling M-mode to switch trigger contexts at privilege mode boundaries, such attacks can be prevented. When* `mdbgen` *is 1, DMODE remains accessible only from Debug Mode.*

If the hart is running in a debug-allowed lower privilege mode (e.g., S/HS-mode) when `mdbgen` is 0:

- Single-stepping cannot stop in M-mode.
- Interrupts to M-mode cannot be disabled by setting `dcsr`.STEPIE=0.



*When* `mdbgen`*=0 and* `dcsr`*.STEP=1, a single-stepped instruction in a debug-allowed privilege mode may transfer control to the M-mode trap handler. The hart will execute the handler in M-mode and re-enter Debug Mode immediately after an MRET instruction returns to the debug-allowed privilege mode (i.e., MRET with* `mstatus`*.MPP<3). The hart does not re-enter Debug Mode if the MRET instruction returns to a debug-disallowed privilege mode (i.e., MRET with* `mstatus`*.MPP=3,* `mdbgen`*=0).*



*This specification assumes that the debugger ensures* `mdbgen` *shall never be set to 0 while the hart is in Debug Mode. Setting* `mdbgen` *to 0 while in Debug Mode could lead to undefined behavior; the hart may lose its debug privileges unexpectedly, potentially causing the debug session to fail or become insecure.*

### 3.1.6. S/HS-mode External Debug Security Extension (Smsedbgsec)

The Smsedbgsec extension is optional and requires Smmedbgsec. It introduces the `SEDBGEN` field in CSR `mdtcfg` and controls S/HS-mode debuggability when M-mode external debug is disallowed.

The `SEDBGEN` field only takes effect when `mdbgen` is 0.

When `SEDBGEN` is set to 1, S/HS-mode external debug is allowed:

- The debug access privilege for the hart is S/HS-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with S/HS-mode privilege.
- The debug operations are allowed when the hart executes in S/HS-mode, U-mode, VS-mode, or VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs`.CMDERR to 3.
- The `sdcsr` and `sdpc` registers (Section 3.1.6.1) are introduced in Smsedbgsec to provide an S/HS-mode debugger with access to `dcsr` and `dpc`.
- The **effective debug access privilege** of loads and stores in Debug Mode may be modified as described in Section 3.1.6.2.
- The allowed resume privilege modes include S/HS-mode and all lower privilege modes.

When `SEDBGEN` is set to 0 and the hart is running in S/HS-mode, external debug is disallowed for S/HS-mode:

- The hart will not enter Debug Mode while running in S/HS-mode:
  - Halt requests will remain pending until external debug is allowed.

  - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
  - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire while in S/HS-mode.
- Accesses to S/HS-mode accessible CSRs (e.g., `sdcsr` and `sdpc`) will fail and `abstractcs`.CMDERR is set to 3 (exception).
- The configuration visible in `sdcsr` will be ignored as if it were 0.
- `DMODE` in trigger state may be read/write from S/HS-mode, as specified in Section 4.3.

When `SEDBGEN` is set to 0 and the hart is running in a debug-allowed lower privilege mode (e.g., U-mode), the following restrictions apply:

- Single-stepping cannot stop in S/HS-mode.
- Interrupts delegated to S/HS-mode cannot be disabled by setting `dcsr`.STEPIE=0.



*When `SEDBGEN`=0 and `sdcsr`.STEP=1, a single-stepped instruction in a debug-allowed privilege mode may transfer control to the S/HS-mode trap handler. The hart will execute the handler in S/HS-mode and re-enter Debug Mode immediately after an SRET instruction returns to the debug-allowed privilege mode (i.e., SRET with `sstatus`.SPP=0). The hart does not re-enter Debug Mode if the SRET instruction returns to a debug-disallowed privilege mode.*

#### `sdcsr` and `sdpc`

When `SEDBGEN` is 1, the `sdcsr` and `sdpc` registers provide S/HS-mode read/write access to the `dcsr` and `dpc` registers, respectively. However, `sdcsr` does not expose access to the `MPRVEN` field; instead, it repurposes the `MPRVEN` bit position as a `DMPRV` field that modifies the **effective debug access privilege** in S/HS-mode. Both registers are accessible only in Debug Mode.

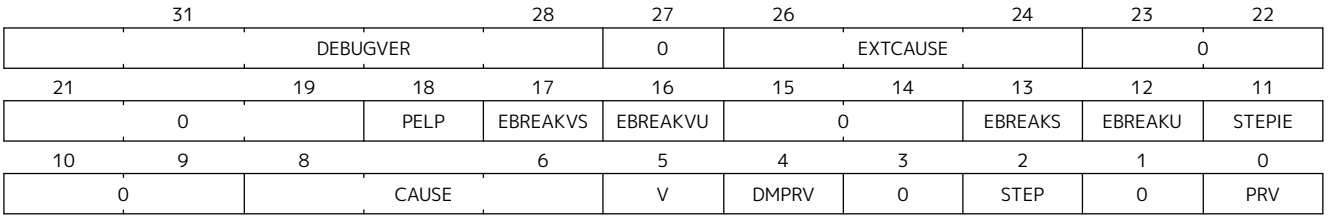
*Table 6. CSR addresses for S/HS-mode shadows of Debug Mode CSRs*

| Number | Name | Description |
|---|---|---|
| address TBD | sdcsr | S/HS-mode debug control and status register. |
| address TBD | sdpc | S/HS-mode debug program counter. |

The `sdcsr` register exposes a subset of `dcsr`, formatted as shown in Register 2, while the `sdpc` register provides full access to `dpc`.



*Unlike `dcsr` and `dpc`, the `dscratch0/1` registers do not have an S/HS-mode access mechanism, and external debuggers with S/HS-mode privilege cannot use them. In The RISC-V Debug Specification [1], `dscratch*` are optional scratch registers that implementations may use internally; `hartinfo`.NSCRATCH indicates how many are available, and a debugger must not write to these registers unless `hartinfo` explicitly mentions them. Compliant debug flows already work without `dscratch` (for example, Access Register, GPRs, and data registers). Sdsec shadows only the required control state at each privilege level (`dcsr`/`dpc` → `sdcsr`/`sdpc` and, when applicable, `udcsr`/`udpc`); no `sdscratch` registers are defined. In an execution-based implementation, the park loop runs at M-mode and may use existing `dscratch` registers, while Abstract Commands and the Program Buffer run at the debug access privilege.*




*Register 2: S/HS-mode debug control and status register (sdcsr)*



*The NMIP, MPRVEN, STOPTIME, STOPCOUNT, EBREAKM, and CETRIG fields in* `dcsr` *are configurable only by M-mode; they are masked in* `sdcsr`*, while PRV[1] is hardwired to 0 in* `sdcsr`*. The field for MPRVEN is reclaimed by DMPRV in the* `sdcsr` *layout to avoid wasting fields.*

`sdcsr`.PELP aliases `dcsr`.pelp. The field is present only when the Zicfilp extension is implemented; otherwise it is read-only 0.

*Table 7. Details of the* `dmprv` *field in* `sdcsr`

| Field | Description | Access | Reset |
|---|---|---|---|
| DMPRV | 0 (normal): The privilege level in Debug Mode is not modified.<br><br>1: In Debug Mode, the privilege level for load and store operations is modified to the effective debug access privilege as described in Section 3.1.6.2 and Section 3.1.7.2. | WARL | 0 |

#### Debug Access Privilege for Memory in Smsedbgsec

The `sdcsr`.DMPRV field takes effect when `mdbgen` is 0 and is read-only 0 when `mdbgen` is 1. When `SEDBGEN` is 1, `sdcsr`.DMPRV can modify the **effective debug access privilege** used by an S/HS-mode debugger's loads and stores in Debug Mode. When `sdcsr`.DMPRV=0, the **effective debug access privilege** of loads and stores in Debug Mode follows Table 4; when `sdcsr`.DMPRV=1, the **effective debug access privilege** of loads and stores in Debug Mode is represented by `sstatus`.SPP and `hstatus`.SPV (if the hypervisor extension is supported).

The `sdcsr`.DMPRV does not affect the virtual-machine load/store instructions, HLV, HLVX, and HSV.

### 3.1.7. VS-mode External Debug Security Extension (Smvsedbgsec)

The Smvsedbgsec extension is optional and requires Smsedbgsec. It introduces the `VSEDBGEN` field in CSR `mdtcfg` and controls VS-mode debuggability when S/HS-mode external debug is disallowed.

The `VSEDBGEN` field only takes effect when both `mdbgen` is 0 and `SEDBGEN` is 0.

When `VSEDBGEN` is set to 1, VS-mode external debug is allowed:

- The debug access privilege for the hart is VS-mode. Abstract Commands, including Quick Access, and instructions in the Program Buffer operate with VS-mode privilege.
- The debug operations are allowed when the hart executes in VS-mode or VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs`.CMDERR to 3.
- The `sdcsr` and `sdpc` registers (from Smsedbgsec) provide VS-mode read/write access to `dcsr` and `dpc`, respectively, as specified in Section 3.1.7.1.

- The **effective debug access privilege** of loads and stores in Debug Mode may be modified as described in Section 3.1.7.2.
- The allowed resume privilege modes include VS-mode and VU-mode.

When `VSEDBGEN` is set to 0 and the hart is running in VS-mode, external debug is disallowed for VS-mode:

- The hart will not enter Debug Mode while running in VS-mode:
  - Halt requests will remain pending until external debug is allowed.
  - Triggers with ACTION=1 (enter Debug Mode) will not match or fire.
  - EBREAK cannot enter Debug Mode and always raises a breakpoint exception.
- The external trigger outputs (with ACTION=8/9) will not match or fire while in VS-mode.
- Accesses to VS-mode accessible CSRs (e.g., `sdcsr` and `sdpc`) will fail and `abstractcs`.CMDERR is set to 3 (exception).
- The configuration visible to VS-mode through `sdcsr` will be ignored as if it were 0.
- `DMODE` in trigger state may be read/write from VS-mode, as specified in Section 4.3.

When `VSEDBGEN` is set to 0 and the hart is running in a debug-allowed VU-mode, the following restrictions apply:

- Single-stepping cannot stop in VS-mode.
- Interrupts delegated to VS-mode cannot be disabled by setting `sdcsr`.STEPIE=0.



*When `VSEDBGEN`=0 and `sdcsr`.STEP=1, a single-stepped instruction in a debug-allowed VU-mode may transfer control to the VS-mode trap handler. The hart will execute the handler in VS-mode and re-enter Debug Mode immediately after an SRET instruction returns to the debug-allowed VU-mode. The hart does not re-enter Debug Mode if the SRET instruction returns to a debug-disallowed VS-mode.*

#### `sdcsr` and `sdpc` in VS-mode

When `VSEDBGEN` is 1, the `sdcsr` and `sdpc` (Section 3.1.6.1) are accessible with VS-mode privilege, providing access to the `dcsr`. The `sdcsr`.EBREAKS and `sdcsr`.EBREAKU fields are redirected to `dcsr`.EBREAKVS and `dcsr`.EBREAKVU, while writes to `sdcsr`.EBREAKVS, `sdcsr`.EBREAKVU, and `sdcsr`.V are discarded (writes are ignored and reads return 0). Similar to `sdcsr` access when `SEDBGEN` is 1, `sdcsr`.DMPRV modifies the **effective debug access privilege** in VS-mode.



*Redirected access to `dcsr`.EBREAKVS and `dcsr`.EBREAKVU unifies the configuration for both S/HS-mode and VS-mode. However, the virtualization mode cannot be changed through `sdcsr`.V by a VS-mode debugger.*

#### Debug Access Privilege for Memory in Smvsedbgsec

The `sdcsr`.DMPRV modifies the **effective debug access privilege** of loads and stores for a VS-mode debugger when `SEDBGEN` is 0 and `VSEDBGEN` is 1.

When `sdcsr`.DMPRV=0, the **effective debug access privilege** of loads and stores in Debug Mode follows Table 4; when `sdcsr`.DMPRV=1, the **effective debug access privilege** of loads and stores in Debug Mode is represented by `vsstatus`.SPP with the virtualization mode being honored as 1.

### 3.1.8. U-mode and VU-mode External Debug Security Extension (Smuedbgsec)

The Smuedbgsec extension is optional. It requires Smmedbgsec and, on harts that implement S/HS-mode, Smsedbgsec. When VU-mode control is implemented, Smuedbgsec also requires Smvsedbgsec. Smvsedbgsec is not required when only U-mode control is implemented and `VUEDBGEN` is read-only 0. Smuedbgsec introduces independent `UEDBGEN` and `VUEDBGEN` fields in CSR `mdtcfg`. If a mode or its corresponding control is not implemented, the corresponding field is read-only 0.

#### U-mode External Debug Control

The `UEDBGEN` field takes effect when `mdbgen`=0 and `SEDBGEN`=0. Because U-mode and VS/VU-mode are not ordered with respect to each other, `VSEDBGEN` and `VUEDBGEN` do not affect U-mode external debug permission.

When `UEDBGEN`=1, `mdbgen`=0, and `SEDBGEN`=0, U-mode external debug is allowed:

- The debug access privilege is U-mode. Abstract Commands and instructions in the Program Buffer operate with U-mode privilege.
- The debug operations are allowed when the hart executes in U-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs`.CMDERR to 3.
- The `udcsr` and `udpc` registers described in Section 3.1.8.3 provide access to the permitted `dcsr` and `dpc` state.
- U-mode is the only allowed resume privilege mode.

When `UEDBGEN` is set to 0 and the hart is running in U-mode, external debug is disallowed for U-mode:

- The hart will not enter Debug Mode while running in U-mode:
  - Halt requests remain pending until external debug is allowed.
  - Triggers with ACTION=1 do not match or fire.
  - EBREAK raises a breakpoint exception instead of entering Debug Mode.
- External trigger outputs with ACTION=8 or ACTION=9 do not match or fire.
- Accesses to user-visible debug CSRs fail and set `abstractcs`.CMDERR to 3.
- The configuration visible through `udcsr` is ignored as if it were 0.

#### VU-mode External Debug Control

The `VUEDBGEN` field takes effect when `mdbgen`=0, `SEDBGEN`=0, and `VSEDBGEN`=0. `UEDBGEN` does not affect VU-mode external debug permission.

When `VUEDBGEN`=1, `mdbgen`=0, `SEDBGEN`=0, and `VSEDBGEN`=0, VU-mode external debug is allowed:

- The debug access privilege is VU-mode. Abstract Commands and instructions in the Program Buffer operate with VU-mode privilege.
- The debug operations are allowed when the hart executes in VU-mode.
- Privilege-changing instructions (other than EBREAK) executed in the Program Buffer must raise an exception, stopping execution and setting `abstractcs`.CMDERR to 3.
- The `udcsr` and `udpc` registers described in Section 3.1.8.3 provide access to the permitted `dcsr` and `dpc`

state.

- VU-mode is the only allowed resume privilege mode.

When `VUEDBGEN` is set to 0 and the hart is running in VU-mode, external debug is disallowed for VU-mode:

- The hart will not enter Debug Mode while running in VU-mode:
  - Halt requests remain pending until external debug is allowed.
  - Triggers with ACTION=1 do not match or fire.
  - EBREAK raises a breakpoint exception instead of entering Debug Mode.
- External trigger outputs with ACTION=8 or ACTION=9 do not match or fire.
- Accesses to user-visible debug CSRs fail and set `abstractcs`.CMDERR to 3.
- The configuration visible through `udcsr` is ignored as if it were 0.

#### `udcsr` and `udpc`

The `udcsr` and `udpc` registers provide U-mode or VU-mode read/write access to the `dcsr` and `dpc` state when the corresponding `UEDBGEN` or `VUEDBGEN` control allows debug. The `udcsr` exposes the subset of `dcsr` shown in Register 3. Access to `udcsr`.EBREAKU is redirected to `dcsr`.EBREAKVU when the virtualization mode is 1. The `udpc` register provides full access to `dpc`.

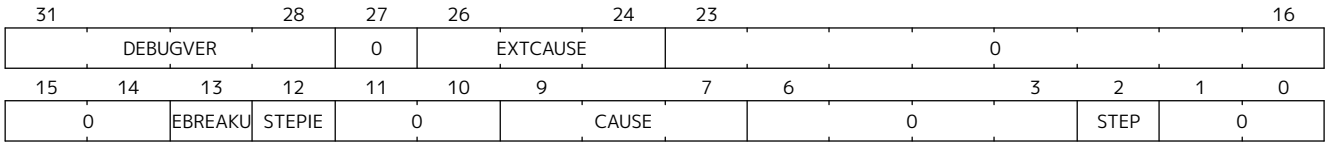


*The `udcsr` does not expose `dcsr`.V. A U-mode or VU-mode debugger therefore cannot switch between U-mode and VU-mode.*

*Table 8. CSR addresses for U/VU-mode shadows of Debug Mode CSRs*

| Number | Name | Description |
|---|---|---|
| address TBD | udcsr | U-mode or VU-mode debug control and status register. |
| address TBD | udpc | U-mode or VU-mode debug program counter. |

The `udcsr` register exposes a subset of `dcsr`, formatted as shown in Register 3, while the `udpc` register provides full access to `dpc`.




*Register 3: U-mode or VU-mode debug control and status register (udcsr)*

## 3.2. Trace Security Extensions

When trace is implemented, trace output can expose sensitive information across privilege boundaries. The extensions below provide security controls to restrict trace output based on privilege levels.

### 3.2.1. Trace Security Control States

The following table defines the trace control states introduced by the trace security extensions. In this specification, *control state* covers both platform-controlled states, such as `mtrcen`, and CSR fields, such as `mdtcfg.SETRCEN`.

*Table 9. Trace Control States*

| Extension | Trace Control State | Target Modes |
|---|---|---|
| Smmetrcsec | mtrcen | M-mode |
| Smsetrcsec | mdtcfg.SETRCEN | S/HS-mode |
| Smvsetrcsec | mdtcfg.VSETRCEN | VS-mode |
| Smuetrcsec | mdtcfg.UETRCEN, mdtcfg.VUETRCEN | U-mode and VU-mode, independently |

Allowing trace for a privilege mode also allows it for all lower privilege modes. U-mode and VU-mode are not ordered with respect to each other, so `UETRCEN` and `VUETRCEN` control them independently. Each `mdtcfg` control field introduced by an unimplemented optional trace extension is read-only 0.



*The availability of trace output is controlled through signals defined in the RISC-V hart-trace interface (RHTI) [4]. An implementation can convert the logical* `sec_inhibit` *signal to the canonical trace interface signals.*

The following `mdtcfg` fields provide trace control states for privilege modes other than M-mode. The `mdtcfg` CSR must be implemented when Smmetrcsec is implemented, even if Smmedbgsec is not implemented.

*Table 10. Details of trace security control states in* `mdtcfg`

| Field | Description | Access | Reset |
|---|---|---|---|
| SETRCEN | S/HS-mode external trace enable. When set to 1, trace is allowed for S/HS-mode, U-mode, VS-mode, and VU-mode when `mtrcen` is 0. Introduced by Smsetrcsec. | WARL | 0 |
| VSETRCEN | VS-mode external trace enable. When set to 1, trace is allowed for VS-mode and VU-mode when `mtrcen`=0 and `SETRCEN`=0. This field does not control U-mode. Introduced by Smvsetrcsec. | WARL | 0 |
| UETRCEN | U-mode external trace enable. When set to 1, U-mode trace is allowed when `mtrcen`=0 and `SETRCEN`=0. `VSETRCEN` and `VUETRCEN` do not affect U-mode permission. Introduced by Smuetrcsec. | WARL | 0 |
| VUETRCEN | VU-mode external trace enable. When set to 1, VU-mode trace is allowed when `mtrcen`=0, `SETRCEN`=0, and `VSETRCEN`=0. `UETRCEN` does not affect VU-mode permission. Introduced by Smuetrcsec. | WARL | 0 |

### 3.2.2. M-mode Trace Security Extension (Smmetrcsec)

The Smmetrcsec extension is the base extension of the Secure Trace family.



*The* `mtrcen` *state is a per-hart platform-controlled state that controls whether M-mode trace output is enabled. It may be delivered to the hart through various methods, such as a new input port, a handshake with the system Root of Trust (RoT), or other methods. The* `mtrcen` *state for the Root-of-Trust (RoT) itself should be managed by SoC hardware, likely dependent on lifecycle fusing. The implementation can choose to group several harts together and use one signal to drive their* `mtrcen` *state or assign each hart its own dedicated state. For example, a homogeneous computing system can use a signal to drive all* `mtrcen` *states to enforce a unified trace policy across all harts. Unlike external debug, trace has no privilege mode above M-mode, so this specification does not introduce a platform-level enable analogous to* `pdbgsecen`*. The* `pdbgsecen` *state does not control trace.*

When `mtrcen` is set to 1, trace is allowed for all privilege modes. When `mtrcen` is set to 0, trace is inhibited

for M-mode by asserting the logical `sec_inhibit` signal to the trace encoder when the hart is executing in M-mode.

### 3.2.3. S/HS-mode Trace Security Extension (Smsetrcsec)

The Smsetrcsec extension is optional and requires Smmetrcsec. It introduces the `SETRCEN` field in CSR `mdtcfg` and controls S/HS-mode trace output when M-mode trace is disallowed.

The `SETRCEN` field only takes effect when `mtrcen` is 0.

When `SETRCEN` is set to 1, trace is allowed for all privilege modes except M-mode. When `SETRCEN` is set to 0, trace is inhibited for S/HS-mode by asserting the logical `sec_inhibit` signal to the trace encoder when the hart is executing in S/HS-mode.

### 3.2.4. VS-mode Trace Security Extension (Smvsetrcsec)

The Smvsetrcsec extension is optional and requires Smsetrcsec. It introduces the `VSETRCEN` field in CSR `mdtcfg` and controls VS-mode trace output when S/HS-mode trace is disallowed.

The `VSETRCEN` field only takes effect when both `mtrcen` is 0 and `SETRCEN` is 0.

When `VSETRCEN` is set to 1, trace is allowed for VS-mode and VU-mode. When `VSETRCEN` is set to 0, trace is inhibited for VS-mode by asserting the logical `sec_inhibit` signal to the trace encoder when the hart is executing in VS-mode.

### 3.2.5. U-mode and VU-mode Trace Security Extension (Smuetrcsec)

The Smuetrcsec extension is optional. It requires Smmetrcsec and, on harts that implement S/HS-mode, Smsetrcsec. When VU-mode trace control is implemented, Smuetrcsec also requires Smvsetrcsec. Smvsetrcsec is not required when only U-mode trace control is implemented and `VUETRCEN` is read-only 0. Smuetrcsec introduces independent `UETRCEN` and `VUETRCEN` fields in CSR `mdtcfg`. If a mode or its corresponding trace control is not implemented, the corresponding field is read-only 0.

The `UETRCEN` field takes effect when `mtrcen`=0 and `SETRCEN`=0. `VSETRCEN` and `VUETRCEN` do not affect U-mode trace permission. When `UETRCEN` is 1, trace is allowed in U-mode; when it is 0, trace is inhibited in U-mode by asserting the logical `sec_inhibit` signal.

The `VUETRCEN` field takes effect when `mtrcen`=0, `SETRCEN`=0, and `VSETRCEN`=0. `UETRCEN` does not affect VU-mode trace permission. When `VUETRCEN` is 1, trace is allowed in VU-mode; when it is 0, trace is inhibited in VU-mode by asserting the logical `sec_inhibit` signal.

# Chapter 4. Smtdeleg and Sstcfg (ISA Extensions)

Sdtrig [1] CSRs are accessible only with M-mode privilege. A self-hosted debugger with less than M-mode privilege must use an SBI, which adds a trap to every trigger access. An external debugger cannot call an SBI from Debug Mode, so when its debug access privilege is less than M-mode, it cannot access triggers.

The Smtdeleg extension lets S/HS-mode and VS-mode access Sdtrig triggers through the indirect CSR interface (Sscsrind) [5], and controls that access with the state-enable extensions (Smstateen/Ssstateen) [6]. Sstcfg comprises the supervisor components of Smtdeleg. Smtdeleg and Sstcfg are not members of the Secure Debug or Secure Trace families.



*Trigger access delegation is all-or-none: the state-enable controls do not select individual triggers. When external debug is enabled, software shall not delegate trigger access to a mode less privileged than the debug access privilege. This leaves the mode at the debug access privilege able to manage triggers for both external and self-hosted debug. When software delegates trigger access, it is expected to provide a clean trigger state for the delegated mode, which then has full control over that state. Sharing trigger state between the delegating and delegated modes, or between contexts managed by the delegating mode, is entirely software-managed. If software chooses to share trigger state, it is responsible for saving the state before switching away from a context and restoring it when that context resumes.*

## 4.1. Supervisor and Virtual Supervisor Trigger Access

While `siselect` holds the *trigger access value*, `sireg5` provides supervisor read/write access to the trigger selection index, and `sireg` through `sireg4` provide supervisor read/write access to the state of the selected trigger, the same state accessed by M-mode CSRs `tdata*` and `tinfo`. The trigger access value is to be allocated in the `siselect` select space.

*Table 11. Indirect Trigger Register Mappings When `siselect` Holds the Trigger Access Value*

| Indirect CSR | State Accessed |
|---|---|
| `sireg` | Same as `tdata1` |
| `sireg2` | Same as `tdata2` |
| `sireg3` | Same as `tdata3` |
| `sireg4` | Same as `tinfo` |
| `sireg5` | Same as `tselect` |

Similarly, when `vsiselect` holds the trigger access value, the `vsireg*` registers provide access to trigger state in the same manner as described above for `sireg*`.

Attempts to access `sireg6` while `siselect` holds the trigger access value, or `vsireg6` while `vsiselect` holds the trigger access value, raise an illegal-instruction exception.



*Trigger interfaces follow the normal CSR privilege and virtualization rules. Software may use any accessible interface, subject to that interface's field mappings and access controls. For example, the S/HS-mode can access trigger state through `sireg*` and `vsireg*` while `siselect` and `vsiselect` hold the trigger access value.*

## 4.2. Trigger State Access Modifications

`sireg*` and `vsireg*` provide access to a subset of the fields in the native trigger registers, and in some cases

fields are relocated. The field modifications that apply when trigger registers are accessed via **sireg*** or **vsireg*** are documented below.

*Table 12. Delegated Trigger CSR Field Modifications*

| Sdtrig Register | **sireg*** Field Modifications | **vsireg*** Field Modifications |
|---|---|---|
| **mcontrol6**, **icount**, **etrigger**, **itrigger**<br><br>(accessed by **tdata1**) | The M field of **sireg** is read-only 0 | The S field of **vsireg** accesses **tdata1**.VS<br>The U field of **vsireg** accesses **tdata1**.VU<br>The M, VS, and VU fields of **vsireg** are read-only 0 |
| **textra32**, **textra64**<br><br>(accessed by **tdata3**) | none | MHVALUE and MHSELECT are read-only 0 |



*Because the delegating mode is expected to provide a clean trigger state before delegation, fields marked read-only 0 are effectively zero when accessed by the delegated mode.*

The trigger types supported through **sireg*** and **vsireg*** are **mcontrol6**, **icount**, **etrigger**, **itrigger**, and disabled (TYPE=15). If the selected trigger has any other type, accesses to its trigger state read 0 and writes are ignored.

## 4.3. DMODE

When **tdata1**.DMODE=1 for the selected trigger and the hart is not in Debug Mode, **sireg** through **sireg4** and **vsireg** through **vsireg4** read 0 and writes are ignored while their corresponding select register holds the trigger access value, unless the access is from:

- M-mode with **mdbgen**=0 (Section 3.1.5).
- S/HS-mode with **mdbgen**=0 and **SEDBGEN**=0.
- VS-mode with **mdbgen**=0, **SEDBGEN**=0, **UEDBGEN**=0, and **VSEDBGEN**=0.

For accesses meeting these conditions, **DMODE** and trigger state are readable and writable through any accessible trigger interface. VS-mode accesses to **sireg** through **sireg4** are substituted by Sscsrind to **vsireg** through **vsireg4**. The field mappings and access controls of the interface used still apply.



*Allowing read/write when **DMODE**=1 is to enable trigger context switching. Saving and restoring trigger context is required for policy isolation when the applicable Secure Debug and trigger delegation extensions are implemented, triggers are delegated to the directly less-privileged implemented mode, external debug is allowed for that mode in at least one context, and switching contexts changes the external debug policy.*

*The software responsible for switching that trigger context is:*

- *M-mode for S/HS-mode, or for U-mode when S-mode is not implemented.*
- *S/HS-mode for U-mode and VS-mode.*
- *VS-mode for VU-mode.*

*When the external debug policy is uniform across those contexts, trigger save and restore can be skipped for policy isolation.*

## 4.4. Supervisor and Virtual Supervisor Trigger State Access Control

When Smstateen is implemented, `mstateen0`.TR=1 delegates trigger access to S/HS-mode. If S-mode is not implemented, it instead delegates trigger access to U-mode through Sutcfg (Section 6.3). When `mstateen0`.TR=0, S/HS-mode accesses to `sireg` through `sireg5` or `vsireg` through `vsireg5` raise an illegal-instruction exception while the corresponding select register holds the trigger access value.

If the H extension is implemented, `hstateen0`.TR=1 delegates trigger access from HS-mode to VS-mode, provided `mstateen0`.TR=1. VS-mode accesses to `sireg` through `sireg5` are substituted by Sscsrind to `vsireg` through `vsireg5`. While `vsiselect` holds the trigger access value, these accesses raise an illegal-instruction exception if `mstateen0`.TR=0, or a virtual-instruction exception if `mstateen0`.TR=1 and `hstateen0`.TR=0. `hstateen0`.TR is read-only 0 when `mstateen0`.TR=0.

These enables remain subject to the CSR privilege rules, other applicable state-enable controls, and the `DMODE` rules in Section 4.3.

The TR bit is bit 3 in `mstateen0` and `hstateen0`. The combination of Sstcfg and Ssstateen incorporates the TR bit of `hstateen0` but not that of `mstateen0`, since machine-level CSRs are not visible to supervisor level.



*See the Sscsrind specification [5] for how bit 60 (CSRIND) in* `mstateen0` *and* `hstateen0` *can also restrict access to* `siselect`/`sireg*` *and* `vsiselect`/`vsireg*` *from privilege modes less privileged than M-mode.*

## 4.5. Dependencies

Smtdeleg depends on Sdtrig, Smcsrind/Sscsrind [5], and Smstateen/Ssstateen [6]. `hstateen0`.TR exists only when the H extension is implemented. Sstcfg requires Smtdeleg.

## 4.6. Summary of New State

Smtdeleg and Sstcfg add no CSRs. They allocate the trigger access value in the `siselect` select space and the following state-enable bits.

*Table 13. New Smstateen Bits Introduced by Smtdeleg*

| Register | Bit | Purpose |
|---|---|---|
| `mstateen0` | 3 (TR) | Controls trigger access below M-mode |
| `hstateen0` | 3 (TR) | Controls trigger access when V=1 |

# Chapter 5. Sucsrind (ISA Extension)

This chapter specifies the Sucsrind ISA extension, which extends the indirect CSR access mechanism defined by Smcsrind [5] to U-mode. Sucsrind is not a member of the Secure Debug or Secure Trace families.

Sucsrind provides U-mode with its own set of indirect register select and alias CSRs. This enables extensions to expose register arrays to U-mode software without consuming large portions of the limited user-level CSR address space, mirroring the benefits Sscsrind provides at supervisor level.



*The Sscsrind specification reserves CSR address space for a possible future Sucsrind extension that extends indirect CSR access to user mode.*

Sucsrind requires Smcsrind, which provides the machine-level indirect CSR interface and the machine-level behavior for the hart. When S-mode is implemented, Sucsrind also requires Sscsrind.

## 5.1. User-level CSRs

*Table 14. User Indirect CSR Access CSRs*

| Number | Privilege | Width | Name | Description |
|---|---|---|---|---|
| address TBD | URW | XLEN | `uiselect` | User indirect register select |
| address TBD | URW | XLEN | `uireg` | User indirect register alias |
| address TBD | URW | XLEN | `uireg2` | User indirect register alias 2 |
| address TBD | URW | XLEN | `uireg3` | User indirect register alias 3 |
| address TBD | URW | XLEN | `uireg4` | User indirect register alias 4 |
| address TBD | URW | XLEN | `uireg5` | User indirect register alias 5 |
| address TBD | URW | XLEN | `uireg6` | User indirect register alias 6 |

The value of `uiselect` determines which register is accessed upon read or write of each of the user indirect alias CSRs (`uireg*`). `uiselect` value ranges are allocated to dependent extensions, which specify the register state accessible via each `uireg*` register, for each `uiselect` value. `uiselect` is a WARL register.

The `uiselect` register will support the value range 0..0xFFF at a minimum. A future extension may define a value range outside of this minimum range. Only if such an extension is implemented will `uiselect` be required to support larger values.



*Requiring a range of 0-0xFFF for `uiselect`, even though most or all of the space may be reserved or inaccessible, permits S-mode to emulate indirectly accessed registers in this implemented range, including registers that may be standardized in the future.*

Values of `uiselect` with the most-significant bit set (bit XLEN-1 = 1) are designated only for custom use. Values of `uiselect` with the most-significant bit clear are designated only for standard use and are reserved until allocated to a standard architecture extension. If XLEN is changed, the most-significant bit of `uiselect` moves to the new position, retaining its value from before.

The behavior upon accessing `uireg*` from M-mode, S-mode, or U-mode, while `uiselect` holds a value that is not implemented at user level, is UNSPECIFIED.



*It is expected that implementations will typically raise an illegal-instruction exception for such accesses, so that, for example, they can be identified as software bugs.*

Attempts to access `uireg*` while `uiselect` holds a number in an allocated and implemented range results in a specific behavior that, for each combination of `uiselect` and `uireg*`, is defined by the extension to which the `uiselect` value is allocated.



*Ordinarily, each* `uireg*` *will access register state, access read-only 0 state, or raise an illegal-instruction exception.*

User-level, supervisor-level, and machine-level select values are separate address spaces from each other. An extension may define user-level and supervisor-level CSRs with the same select value as partial or full aliases, typically for state that can be delegated from supervisor-level to user-level.

The widths of `uiselect` and `uireg*` are always the current XLEN.

## 5.2. Virtualization

Sucsrind does not define separate virtual-user `vu*` indirect CSRs.

When V=1, VS-mode and VU-mode access the same `uiselect` and `uireg*` as U-mode. Higher privilege modes are expected to swap the content of these registers as needed.

## 5.3. Access Control by the State-Enable CSRs

If extension Smstateen is implemented together with Sucsrind, bit 4 (UCSRIND) of state-enable register `mstateen0` controls access to `uiselect` and `uireg*`. When `mstateen0`.UCSRIND=0, an attempt to access `uiselect` or `uireg*` from a privilege mode less privileged than M-mode results in an illegal-instruction exception. Bit 60 (CSRIND) of `mstateen0` continues to control only the Sscsrind CSRs (`siselect`, `sireg*`, `vsiselect`, and `vsireg*`) and does not gate `uiselect` or `uireg*`.

If the hypervisor extension is implemented, `hstateen0`.UCSRIND also controls accesses from VS-mode or VU-mode to `uiselect` and `uireg*`. When `hstateen0`.UCSRIND=0 and `mstateen0`.UCSRIND=1, all attempts from VS-mode or VU-mode to access `uiselect` or `uireg*` raise a virtual-instruction exception.

When S-mode is implemented, `sstateen0`.UCSRIND controls accesses from U-mode and VU-mode to `uiselect` and `uireg*`. Allocating the enable at bit 4 keeps it in the low end of `sstateen0`, with the same bit position in `mstateen0` and `hstateen0`, as required for state accessible at user level [6]. The combination of Sucsrind and Ssstateen incorporates the UCSRIND bit of `hstateen0` and `sstateen0` but not that of `mstateen0`, since machine-level CSRs are not visible to supervisor level.



*Individual extensions that use* `uiselect`/`uireg*` *to access specific register state may define additional* `sstateen` *bits to provide S-mode with fine-grained control over U-mode access to that state. Sutcfg defines* `sstateen0`*.TR to control U-mode access to trigger registers accessed through* `uireg*`.

## 5.4. Dependencies

Sucsrind depends on the following extensions:

- Smcsrind — Machine-level indirect CSR access framework [5].
- Sscsrind (required when S-mode is implemented) — Supervisor-level indirect CSR access framework.
- S-mode (optional) — Required for `sstateen0` access control and for Sscsrind. In implementations without S-mode, `mstateen0` alone controls U-mode access to `uiselect`/`uireg*`.

- H extension (optional) — Required for VS/VU-mode virtualization support.

## 5.5. Summary of New State

*Table 15. New CSRs Introduced by Sucsrind*

| CSR | Address | Privilege | Notes |
|---|---|---|---|
| `uiselect` | address TBD | URW | User indirect register select |
| `uireg` | address TBD | URW | User indirect register alias |
| `uireg2` | address TBD | URW | User indirect register alias 2 |
| `uireg3` | address TBD | URW | User indirect register alias 3 |
| `uireg4` | address TBD | URW | User indirect register alias 4 |
| `uireg5` | address TBD | URW | User indirect register alias 5 |
| `uireg6` | address TBD | URW | User indirect register alias 6 |

*Table 16. New Smstateen Bits Introduced by Sucsrind*

| Register | Bit | Purpose |
|---|---|---|
| `mstateen0` | 4 (UCSRIND) | Controls access to `uiselect` and `uireg*` below M-mode |
| `hstateen0` | 4 (UCSRIND) | Controls access to `uiselect` and `uireg*` in VS/VU-mode when V=1 |
| `sstateen0` | 4 (UCSRIND) | Controls U-mode access to `uiselect` and `uireg*` |

# Chapter 6. Sutcfg (ISA Extension)

The Sutcfg extension lets U-mode and VU-mode access Sdtrig [1] triggers through the user-level indirect CSR interface defined by Sucsrind. Sutcfg requires Sucsrind. When S-mode is implemented, Sutcfg also requires Smtdeleg and Sstcfg. Sutcfg is not a member of the Secure Debug or Secure Trace families.



*Trigger access delegation to U-mode and VU-mode is managed by the higher-privileged mode, as described in the note in Chapter 4.*

## 6.1. User Trigger Access

The `uiselect` value for trigger access is the trigger access value defined in Section 4.1. While `uiselect` holds the trigger access value, `uireg` through `uireg5` access the same state as `sireg` through `sireg5` in Section 4.1, with the field modifications below. Attempts to access `uireg6` while `uiselect` holds the trigger access value raise an illegal-instruction exception.

*Table 17. User Trigger CSR Field Modifications*

| Sdtrig Register | Accesses with V=0 | Accesses with V=1 |
|---|---|---|
| `mcontrol6`, `icount`<br><br>(accessed by `tdata1`) | The M, S, VS, and VU fields of `uireg` are read-only 0 | The U field of `uireg` accesses `tdata1`.VU<br>The M, S, VS, and VU fields of `uireg` are read-only 0 |
| `textra32`, `textra64`<br><br>(accessed by `tdata3`) | MHVALUE, MHSELECT, SVALUE, SSELECT, and SBYTEMASK are read-only 0 | MHVALUE, MHSELECT, SVALUE, SSELECT, and SBYTEMASK are read-only 0 |

The trigger types supported through `uireg*` are `mcontrol6`, `icount`, and disabled (TYPE=15). If the selected trigger has any other type, accesses to its trigger state read 0 and writes are ignored.



`itrigger` *and* `etrigger` *are not available through* `uireg*`*. Sdtrig fires them just before the first instruction of a trap handler, and U-mode and VU-mode do not execute trap handlers.*

## 6.2. DMODE

While `uiselect` holds the trigger access value, `DMODE` and trigger state are readable and writable through `uireg` through `uireg4`, even when the hart is not in Debug Mode, for:

- M-mode accesses with `mdbgen`=0.
- S/HS-mode accesses with `mdbgen`=0 and `SEDBGEN`=0.
- VS-mode accesses with `mdbgen`=0, `SEDBGEN`=0, `UEDBGEN`=0, and `VSEDBGEN`=0.

For all other accesses when the hart is not in Debug Mode, if the selected trigger has `tdata1`.DMODE=1, these registers read 0 and writes are ignored. All accesses remain subject to the field modifications in Section 6.1 and the access control in Section 6.3.

## 6.3. User Trigger State Access Control

When Smstateen is implemented and `uiselect` holds the trigger access value, access to trigger state through `uireg` through `uireg5` is enabled when **all** TR bits applicable to the accessing privilege mode are 1:

*Table 18. Required TR Enables for the User Trigger Interface*

| Accessing Privilege Mode | TR Bits Required to Be 1 |
|---|---|
| M-mode | None |
| S/HS-mode | `mstateen0`.TR |
| VS-mode | `mstateen0`.TR and `hstateen0`.TR |
| U-mode, with S-mode implemented | `mstateen0`.TR and `sstateen0`.TR |
| U-mode, without S-mode | `mstateen0`.TR |
| VU-mode | `mstateen0`.TR, `hstateen0`.TR, and `sstateen0`.TR |

The applicable TR bits are checked in machine, hypervisor, then supervisor order. If a checked bit is 0, the access raises an illegal-instruction exception for `mstateen0`, a virtual-instruction exception for `hstateen0`, or, for `sstateen0`, an illegal-instruction exception from U-mode and a virtual-instruction exception from VU-mode.

When S-mode is not implemented, `mstateen0`.TR alone controls U-mode access. `mstateen0`.TR and `hstateen0`.TR are defined in Chapter 4. The TR bit is bit 3 in `sstateen0`. The combination of Sutcfg and Ssstateen incorporates the TR bit of `sstateen0`.

## 6.4. Summary of New State

Sutcfg adds no CSRs. It uses the trigger access value in the `uiselect` select space and the following state-enable bit. When S-mode is not implemented, Sutcfg also defines `mstateen0`.TR (bit 3), as specified in Chapter 4.

*Table 19. New Smstateen Bits Introduced by Sutcfg*

| Register | Bit | Purpose |
|---|---|---|
| `sstateen0` | 3 (TR) | Controls U-mode and VU-mode trigger access |

# Chapter 7. Debug Module Security (non-ISA) Extension

This chapter defines the required security enhancements for the Debug Module. The debug operations listed below are modified by this extension.

- Halt
- Reset
- Keepalive
- Abstract commands (Access Register, Quick Access, Access Memory)
- System bus access

If any hart in the system implements Smmedbgsec, the Debug Module must also implement the Debug Module Security Extension.

## 7.1. External Debug Security Extensions Discovery

The ISA and non-ISA external debug security extensions impose security constraints and introduce non-backward-compatible changes. The presence of the extensions can be determined by polling the ALLSECURED and/or ANYSECURED bits in `dmstatus` Table 20. These bits will read as 0 when `pdbgsecen` is 0, regardless of whether the harts implement Smmedbgsec. When `pdbgsecen` is 1, ALLSECURED is set to 1 if all currently selected harts implement Smmedbgsec, and ANYSECURED is set to 1 if any currently selected hart implements Smmedbgsec. When any hart implements Smmedbgsec, it indicates that the Debug Module implements the Debug Module Security Extension as described in this chapter.

## 7.2. Halt

The halt behavior for a hart is detailed in Section 3.1. According to *The RISC-V Debug Specification* [1], a halt request must be responded to within one second. However, this constraint must be relaxed, as the request might remain pending when debugging is disallowed. In the case of a halt-on-reset request (SETRESETHALTREQ), the request is only acknowledged by the hart once it has reached a privilege level in which debug is allowed.

## 7.3. Reset

The HARTRESET operation resets selected harts. When M-mode debugging is disallowed, the hart will raise a security fault error to the Debug Module on HARTRESET operations. The error can be detected through ALLSECFAULT and ANYSECFAULT in `dmstatus`.

The NDMRESET operation is a system-level reset not tied to hart privilege levels and resets the entire system (excluding the Debug Module). The Debug Module Security Extension makes the NDMRESET field read-only 0 when `pdbgsecen` is 1. The debugger can determine support for the NDMRESET operation by setting the field to 1 and subsequently verifying the returned value upon reading.

## 7.4. Keepalive

The SETKEEPALIVE bit serves as an optional request for the hart to remain available for debugging. This bit only takes effect when M-mode debugging is allowed; otherwise, the hart behaves as if the bit is not set.



*Keepalive affects whole-hart power and availability, including M-mode. Halt and single-step requests with* `mdbgen`*=0 remain privilege-scoped and do not imply hart-global power control.*

> *Therefore, a lower-privilege debug enable does not grant the debugger authority to keep the hart available through keepalive, and* `SETKEEPALIVE` *remains read-only 0 when* `mdbgen`*=0. If keepalive is needed later, it should be introduced as a separate M-mode delegation extension rather than implied by* `mdtcfg.*EN`*.*

## 7.5. Abstract Commands

The hart's response to abstract commands is detailed in Section 3.1. The following subsection delineates the constraints when the Debug Module issues an abstract command.

### 7.5.1. Relaxed Permission Check `relaxedpriv`

The `relaxedpriv` field is hardwired to 0.

### 7.5.2. Address Translation AAMVIRTUAL

The field `command`.AAMVIRTUAL determines whether the Access Memory command uses a physical or virtual address. When `mdbgen`=1, the behavior follows the original RISC-V Debug Specification [1]. When `mdbgen`=0:

- If AAMVIRTUAL=0, the hart responds with a security fault error (setting `abstractcs`.CMDERR to 6).
- If AAMVIRTUAL=1, addresses are treated as virtual and are translated and protected according to the **debug access privilege**, as defined in Section 3.1.3, and may be modified to the **effective debug access privilege** as defined in Section 3.1.6.2 or Section 3.1.7.2.

### 7.5.3. Quick Access

When M-mode debugging is disallowed (`mdbgen`=0) for a hart, any Quick Access operation will be discarded by the Debug Module, causing `abstractcs`.CMDERR to be set to 6.

> *Quick Access abstract commands initiate a halt, execute the Program Buffer, and resume the selected hart. These halts cannot remain pending until debugging is allowed because the debugger blocks while waiting for completion. To address this, the hart would need to distinguish between Quick Access and other halt requests. Since Quick Access is an optional optimization, the Debug Module Security Extension avoids this additional hardware complexity by disallowing Quick Access when* `mdbgen` *is 0.*

## 7.6. System Bus Access

The System Bus Access must be checked by bus initiator protection mechanisms such as IOPMP [7] or RISC-V Worlds [8]. The bus protection unit can return an error to the Debug Module on illegal accesses; in that case, the Debug Module will set `sbcs`.SBERROR to 6 (security fault error).

> *Trusted entities like the RoT should configure IOPMP or equivalent protection before external debug is allowed.*

## 7.7. Security Fault Error Reporting

A dedicated error code, security fault error (CMDERR 6), is included in `abstractcs`.CMDERR. Issuance of abstract commands under disallowed circumstances sets `abstractcs`.CMDERR to 6. Additionally, the bus

security fault error (SBERROR 6) is introduced in `sbcs`.SBERROR to denote errors related to system bus access.

Error status bits are internally maintained for each hart. The ALLSECFAULT and ANYSECFAULT fields in `dmstatus` indicate the error status of the currently selected harts. These error statuses are sticky and can only be cleared by writing 1 to `dmcs2`.ACKSECFAULT Table 21.

## 7.8. Platform Debug Security Enable

The platform state element `pdbgsecen` is introduced to enable the debug security constraints defined by this specification. When `pdbgsecen` is 0, the platform retains full debugging capabilities, as if the Secure Debug extensions were not implemented:

- All debug operations are executed with M-mode privilege (equivalent to having `mdbgen` set to 1) for all harts in the system.
- The NDMRESET operation is allowed.
- The RELAXEDPRIV field may be configurable.
- System Bus Access may bypass the bus initiator protections.

If `mdtcfg` is implemented, it remains accessible to M-mode software when `pdbgsecen` is 0. Implemented debug-related fields retain their specified WARL behavior, but their values do not constrain external debug; the unrestricted debug behavior defined above applies. The trace-related fields remain independent of `pdbgsecen` and retain their specified effect.

When `pdbgsecen` is 1, the debug security constraints in this specification apply.

## 7.9. Authorization Handoff

An **authorization handoff** occurs when the platform ends debug for one principal or domain, or admits a different one. That includes one S-mode domain finishing debug before another starts.

A hart entering or leaving a debug-allowed privilege mode is not a handoff, as long as the same principal or domain still holds debug authority. Changing `pdbgsecen` or a hart's debug access privilege is not a handoff by itself.

After a handoff, the newly admitted debugger must not observe or use Debug Module state obtained under authorization it does not have. An operation issued before the handoff must not complete afterwards in a way that makes such state observable or usable.

The platform must complete sanitization before the succeeding debugger is admitted. The mechanism is implementation-specific.



*The following state needs an explicit hardware or software clear/invalidate path across a handoff:*

- *Must sanitize:* `data*`; `progbuf*` *if it is DMI-readable;* `sbdata*`.
- *Should also review:* `sbaddress*` *(last SBA address);* `command` *(e.g.* `regno`*);* `abstractauto`*; and the SBA auto bits in* `sbcs` *(*`sbreadondata`, `sbreadonaddr`, `sbautoincrement`*). Leftover control can re-issue an earlier access or reveal what was touched.*

*Clearing existing register contents is insufficient if an earlier operation can later repopulate them.*

> *One option is to set* `dmcontrol`*.DMACTIVE to 0 and wait for the state change to complete, which resets Debug Module state. NDMRESET does not reset the Debug Module. Software sanitization is another option. Writing zero to the affected registers is not a generic sanitization procedure: when* `abstractauto` *is set, accessing* `data` *or* `progbuf` *may execute* `command`*, and writing* `sbdata0` *starts a system bus write.*

## 7.10. Update of Debug Module Registers

The Debug Module Security Extension introduces new fields in the Debug Module registers. In `dmstatus`, ANYSECURED and ALLSECURED are added at bit 20 and bit 21 respectively, while ANYSECFAULT and ALLSECFAULT are added at bit 25 and bit 26. The ACKSECFAULT field is added to `dmcs2` at bit 12.

*Table 20. Details of newly introduced fields in* `dmstatus`

| Field | Description | Access | Reset |
|---|---|---|---|
| ALLSECURED | The field is 0 when `pdbgsecen` is 0. When `pdbgsecen` is 1, this field is 1 if all currently selected harts implement Smmedbgsec. | R | - |
| ANYSECURED | The field is 0 when `pdbgsecen` is 0. When `pdbgsecen` is 1, this field is 1 if any currently selected hart implements Smmedbgsec. | R | - |
| ALLSECFAULT | The field is 1 when all currently selected harts have raised a security fault due to reset operation | R | - |
| ANYSECFAULT | The field is 1 when any currently selected hart has raised a security fault due to reset operation | R | - |

*Table 21. Detail of ACKSECFAULT in* `dmcs2`

| Field | Description | Access | Reset |
|---|---|---|---|
| ACKSECFAULT | 0 (nop): No effect.<br><br>1 (ack): Clears error status bits for any selected harts. | W1 | - |

# Appendix A: Valid Extension Combinations

This appendix lists valid external debug and trace extension combinations for the supported privilege-mode architectures. The two families are independent, so each table applies only to the family it covers.

## A.1. External Debug Security Extension Combinations

*Table 22. Valid External Debug Extension Combinations*

| Architecture | Valid Debug Extension Combinations |
|---|---|
| M-only | • Smmedbgsec |
| M/U | • Smmedbgsec<br>• Smmedbgsec + Smuedbgsec |
| M/S/U | • Smmedbgsec<br>• Smmedbgsec + Smsedbgsec<br>• Smmedbgsec + Smsedbgsec + Smuedbgsec |
| M/S/VS/VU/U | • Smmedbgsec<br>• Smmedbgsec + Smsedbgsec<br>• Smmedbgsec + Smsedbgsec + Smuedbgsec<br>• Smmedbgsec + Smsedbgsec + Smvsedbgsec<br>• Smmedbgsec + Smsedbgsec + Smvsedbgsec + Smuedbgsec |

## A.2. Trace Security Extension Combinations

Table Table 23 shows valid trace extension combinations for different architectures:

*Table 23. Valid Trace Extension Combinations*

| Architecture | Valid Trace Extension Combinations |
|---|---|
| M-only | • Smmetrcsec |
| M/U | • Smmetrcsec<br>• Smmetrcsec + Smuetrcsec |
| M/S/U | • Smmetrcsec<br>• Smmetrcsec + Smsetrcsec<br>• Smmetrcsec + Smsetrcsec + Smuetrcsec |
| M/S/VS/VU/U | • Smmetrcsec<br>• Smmetrcsec + Smsetrcsec<br>• Smmetrcsec + Smsetrcsec + Smuetrcsec<br>• Smmetrcsec + Smsetrcsec + Smvsetrcsec<br>• Smmetrcsec + Smsetrcsec + Smvsetrcsec + Smuetrcsec |

# Appendix B: Software Debug/Trace Security Policy Management

This section provides guidance on how software at different privilege levels can manage debug/trace security policies.

## B.1. M-mode Management

M-mode software is responsible for managing the `SEDBGEN`, `VSEDBGEN`, `UEDBGEN`, and `VUEDBGEN` fields in the `mdtcfg` CSR for external debug, and the `SETRCEN`, `VSETRCEN`, `UETRCEN`, and `VUETRCEN` fields in the `mdtcfg` CSR for trace. The `mdbgen` and `mtrcen` states are read-only to software and are typically controlled by platform hardware, such as an RoT or lifecycle fuses.

M-mode software should:

1. Initialize debug/trace security policy for lower privilege modes based on security requirements.
2. Provide interfaces for dynamic debug/trace security policy updates (e.g., through SBI calls from lower privilege modes).
3. Save and restore debug/trace security policies when switching between supervisor software contexts.
4. Manage trigger context as described in Trigger context switching when switching external debug policy.
5. When Smtdeleg or Sutcfg is implemented, set `mstateen0`.TR when prepared to delegate trigger access to less privileged modes.

## B.2. S/HS-mode Management

S/HS-mode software is responsible for managing the debug/trace security policy for lower privilege modes:

1. Call M-mode to apply the debug/trace security policy before launching an application or VM.
2. Provide interfaces to handle dynamic debug/trace security policy update requests (e.g., SBI calls from VMs) and call M-mode to apply the security policy accordingly.
3. Save and restore debug/trace security policies when switching between applications or VMs.
4. Manage trigger context as described in Trigger context switching when switching external debug policy.
5. When Sutcfg is implemented, set `sstateen0`.TR when prepared to delegate trigger access to U-mode.

## B.3. VS-mode Management

VS-mode software is responsible for managing the debug/trace security policy for applications in VU-mode:

1. Call S/HS-mode to apply the debug/trace security policy before launching an application.
2. Save and restore debug/trace security policies when switching between applications.
3. Manage trigger context as described in Trigger context switching when switching external debug policy.
4. When Sutcfg is implemented, set `sstateen0`.TR when prepared to delegate trigger access to VU-mode.

# Appendix C: Execution-Based Implementation with Sdsec

In an execution-based implementation, the code executing the "park loop" can always run with M-mode privilege to access memory and CSRs. However, once execution is dispatched to an Abstract Command or the Program Buffer, the privilege level used to access memory and CSRs should be restricted to the debug access privilege.

To achieve this, a Debug-Mode-only state element (e.g., a field in a custom CSR) may be introduced to control the privilege level in Debug Mode. When the state is set to 1, Debug Mode allows M-mode privilege; when cleared to 0, it enforces the debug access privilege. The hardware sets this state to 1 upon entering the park loop and clears it to 0 with the final instruction of the park loop, immediately before execution is transferred to an Abstract Command or the Program Buffer.

# Bibliography

[1] “RISC-V Debug Specification.” [Online]. Available: github.com/riscv/riscv-debug-spec.

[2] “RISC-V Efficient Trace for RISC-V.” [Online]. Available: github.com/riscv-non-isa/riscv-trace-spec.

[3] “RISC-V N-Trace (Nexus-based Trace) Specification.” [Online]. Available: github.com/riscv-non-isa/riscv-nexus-trace.

[4] “RISC-V Hart-Trace Interface (RHTI).” [Online]. Available: github.com/riscv-non-isa/riscv-hart-trace-interface.

[5] “Smcsrind/Sscsrind Indirect CSR Access, Version 1.0.” [Online]. Available: docs.riscv.org/reference/isa/priv/indirect-csr.html.

[6] “Smstateen/Ssstateen Extensions, Version 1.0.” [Online]. Available: docs.riscv.org/reference/isa/priv/smstateen.html.

[7] “RISC-V IOPMP Architecture Specification.” [Online]. Available: github.com/riscv-non-isa/riscv-iopmp.

[8] “RISC-V Worlds.” [Online]. Available: github.com/riscv/riscv-worlds.